Ann Arbor Weather Trends

Qihang Cheng

Content

Background

Goal

Determine if there has been a change in Ann Arbor's climate over the past two decades.

Data

Collected from NOAA (National Oceanic and Atmospheric Administration) shows lowest and highest temperature.

Theory

Used the Least Squares method to calculate coefficients to model our data.

Fitted the data to a linear and sinusoidal

Least Square Method

The objective of least squares is to find the parameters of a model function that will best fit a data set. The fit of a model to a data point is measured by its residual

$$r_i = y_i - f(x_i, \beta).$$

The least-squares method finds the optimal parameter values by minimizing the sum of squared residuals

$$S = \sum_{i=1}^{n} r_i^2.$$

Goal: Minimizing $\|y - X\beta\|^2$

$\vec{y}$ = observed data vector

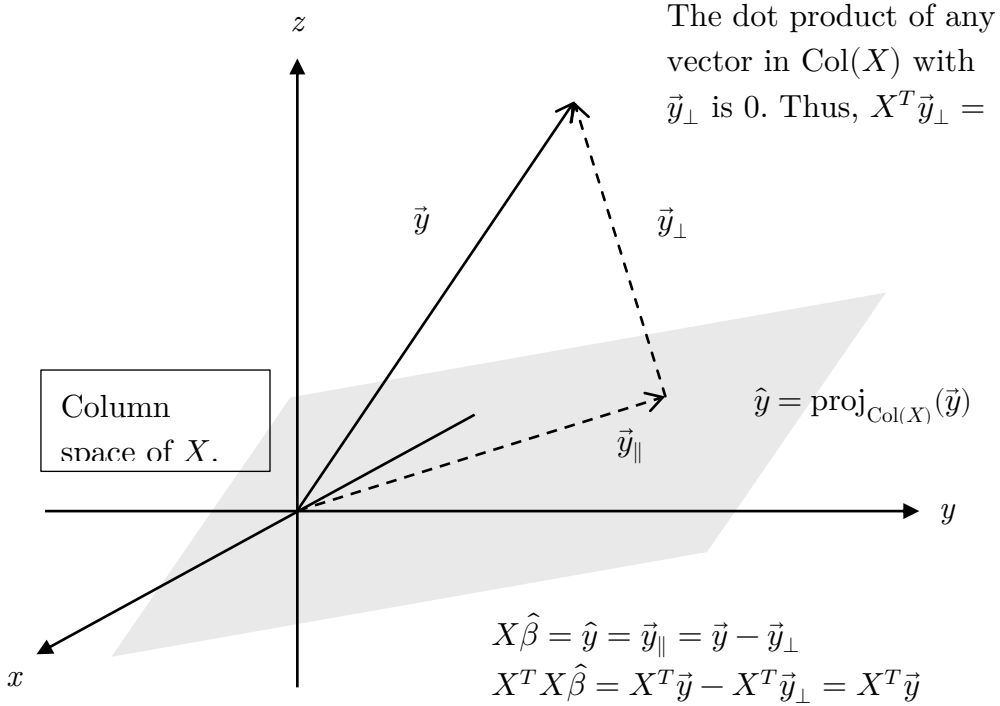
$X\beta$ = predicted data vector

All possible fits $X\beta$ lie in $\text{Col}(X)$.

$\hat{y} \in \text{Col}(X)$

$r = \vec{y} - X\hat{\beta}$

$r \perp \text{Col}(X)$



QR-Decomposition

QR decomposition factors a matrix A into the product $A = QR$, where Q has orthonormal columns and R is an upper triangular matrix.
Although QR decomposition can be computed using the Gram-Schmidt process, the classical Gram-Schmidt algorithm may be numerically unstable for linear least squares problems, especially when the columns of A are nearly linearly dependent. Therefore, more stable methods such as Householder transformations or modified Gram-Schmidt are often preferred.

$$\vec{u}_k = \vec{a}_k - \sum_{j=1}^{k-1} \mathrm{proj}_{\vec{u}_j}(\vec{a}_k)\,, \qquad \vec{e}_k = \frac{\vec{u}_k}{\|\vec{u}_k\|}$$

Goal: Compute The Least Square Using QR Decomposition

$$X = QR$$

$$\begin{bmatrix} | & | & | \\ \vec{a}_1 & \vec{a}_2 & \vec{a}_3 \\ | & | & | \end{bmatrix} = \begin{bmatrix} | & | & | \\ \vec{e}_1 & \vec{e}_2 & \vec{e}_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} \vec{e}_1^T \cdot \vec{a}_1 & \vec{e}_1^T \cdot \vec{a}_2 & \vec{e}_1^T \cdot \vec{a}_3 \\ 0 & \vec{e}_2^T \cdot \vec{a}_2 & \vec{e}_2^T \cdot \vec{a}_3 \\ 0 & 0 & \vec{e}_3^T \cdot \vec{a}_3 \end{bmatrix}$$

Q has orthonormal columns

R is upper triangular

$Q^T Q = \mathrm{I}$

$\hat{y} = X\hat{\beta}$

$Q^T \vec{y} = R\hat{\beta}$

Data Overview

Data had columns for:

- Date
- Avg temp
- High temp
- Low temp
- Inches precipitation
- Inches snow

We only used the date, high temp, and low temp columns.

8392 Rows	3 Variables	20 Years

	Date	TMAX (°F)	TMIN (°F)
3796	7/26/2013	79.0	50.0
1918	7/1/2008	75.0	46.0
922	10/9/2005	57.0	38.0
6160	2/11/2020	34.0	26.0
4242	11/11/2014	59.0	35.0

```
Within 2003-2026,
The overall maximum temperature observed on 9/9/2025.
The overall minimum temperature observed on 1/1/2004.
```

Data Cleaning

In our data, high and low temperatures had no missing values, while avg temp, precipitation and snowfall columns did have missing values. Because high and low temperatures are the most important weather indicators for many people, we excluded the precipitation and snowfall columns, allowing us to avoid having to deal with missing values entirely.

```python
# Data Filtering
def select_date(df: pd.DataFrame) -> pd.DataFrame:
    """
    REQUIRES: df has a 'Date' column with strings or datetime objects.
    MODIFIES: df
    EFFECTS: Converts 'Date' to datetime and returns rows from 2003-04-16 onward.
    """
    df = df.copy()
    df['Date'] = pd.to_datetime(df['Date'])
    return df[df['Date'] >= '2003-04-16']
```

Programming

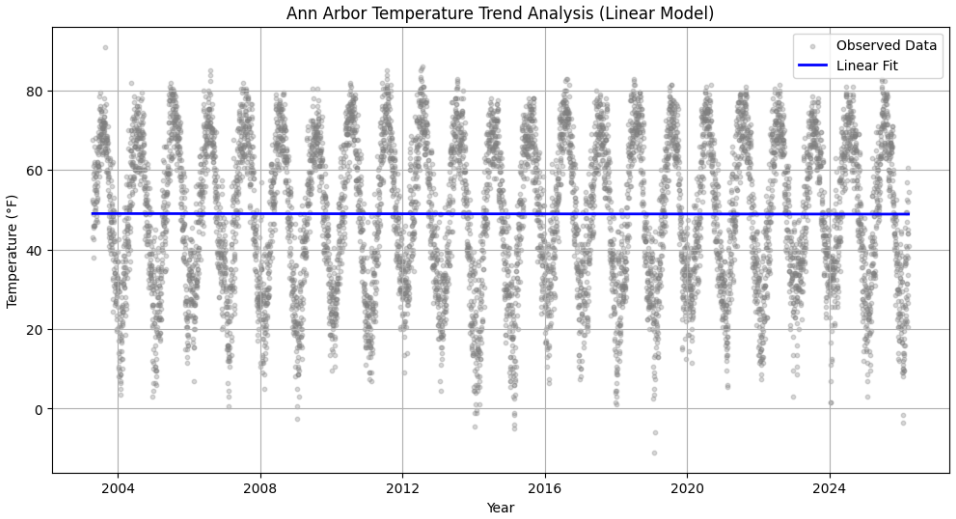
Steps:

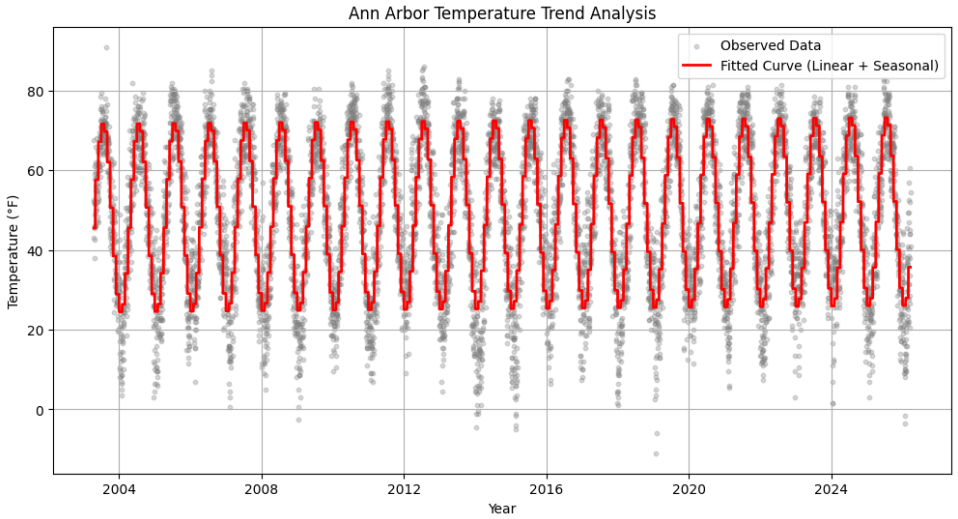
1. Data cleaning & filtering
2. Set up NumPy equations
3. Create normal functions
4. Plotting results
5. Evaluate model

```python
def prepare_features(df: pd.DataFrame) -> np.ndarray:
    # Calculate time.
    start_date = df['Date'].min()
    df['time_days'] = (df['Date'] - start_date).dt.days

    # Convert the time into Months
    df['month'] = df['Date'].dt.month
    df['sin_month'] = np.sin(2 * np.pi * df['month'] / 12)
    df['cos_month'] = np.cos(2 * np.pi * df['month'] / 12)

    # Construct the matrix as [1, time, sin_term, cos_term]
    ones = np.ones(len(df))
    X = np.column_stack((ones,
                         df['time_days'],
                         df['sin_month'],
                         df['cos_month']))
    return X
```

```python
def solve_normal_function(M: np.ndarray, y: np.ndarray) -> np.ndarray:
  # M = Q @ R
  Q, R = np.linalg.qr(M)
  Qty = np.dot(Q.T, y)

  # Solve beta.
  beta = solve_triangular(R, Qty)
  return beta
```

Model Visualization





Linear Model

Intercept	49.063922425340635
Time coefficient	-1.620386416090536e-05
R-squared	0.0000

Sinusoidal Model

Intercept	48.03977137204093
Time coefficient	0.0001984611996935422 8
Sin coefficient	-14.00220819803889
Cos coefficient	-19.169419886721855
R-squared	0.7943

Model Comparison

Model	Linear	Sinusoidal
Equation	$y = (-1.62e-5)x + 49.1$	$y = 48.0 + (1.98e-4)x - 14.0\sin(x) - 19.2\cos(x)$
R^2	0.0000	0.7943
Units x	Days	Months

Conclusion

- A simple linear model fails to capture the temperature pattern.
- Adding sine and cosine seasonal terms greatly improves model fit.
- The seasonal model achieves $R^2 \approx 0.794$.
- Seasonal variation is the strongest driver of temperature changes.
- The positive time coefficient suggests a gradual warming trend, but further analysis is needed.